Developer Manual

COMP3211 Group 149 PIM | Handoff edition | 20 October 2026

The source runs directly as an offline terminal application. The verified development environment is CPython 3.12.6 on macOS 15.7.3, arm64, using Codex and a terminal. Windows is the declared SRS target, but no real Windows run has been performed. The Windows instructions in this manual require the verification checklist before any Windows compatibility claim.

Environment and launch

Use Python 3.12. No pip install, compilation, server, database, web browser or network connection is needed. Do not use document-generation packages as application dependencies: the PDFs were prepared externally and are not imported by the program or its tests.

Open a terminal in the extracted source root, the folder containing main.py:

```
python3.12 --version
python3.12 -X utf8 -B main.py
```

The actual local interpreter reports Python 3.12.6. The first prompt is pim>. The collection starts empty. Try show all, then exit; samples are never loaded automatically. -X utf8 requests UTF-8 text I/O; -B avoids bytecode caches. Use quoted paths inside the application, not operating-system shell commands. Exit uses save/discard/cancel when dirty. Dirty permanent EOF warns on stderr and exits with status 1; clean EOF may exit 0.

Architecture and package responsibilities

Component	Responsibility
main.py	Composition root; creates one shared ExpressionParser, registry, storage, service, command parser, view and controller
model/records.py and validation.py	Four immutable record types and complete input/candidate validation
model/criteria.py and search.py	Six Composite criterion types, symbol-only parser and immutable session definitions
model/service.py and errors.py	Collection mutation, ID history, duplicates, dirty state and named domain exceptions
model/persistence.py	Strict UTF-8 JSON v1 validation and same-directory temporary-file replacement
cli/parser.py and literals.py	Command grammar, immutable carriers and quoted command/path decoding
cli/controller.py and view.py	Confirmation/guard coordination and all terminal I/O/formatting
tests/ and tools/	Automated assertions and the independent model line-coverage runner

Dependencies point from CLI to model. The model never imports CLI or performs terminal I/O. Only the service invokes storage. Expressions are fully parsed before record evaluation; Composite evaluation uses an explicit stack and retains short-circuit semantics. The controller forwards raw expressions and does not duplicate business validation or access model state directly.

Tests and measured model coverage

Run from the source root:

```
python3.12 -B -m unittest discover -s tests -p "test_*.py" -v
python3.12 -B tools/report_model_coverage.py
```

Actual local result on 20 October 2026: 249 test methods passed, zero failures or errors. This comprises 175 model, 57 parser/view/controller, 12 real-process and 5 coverage-tool methods. Tests use temporary directories, controlled failure mocks and fixed dates. They do not modify personal files or require live input.

The second command runs only the eight named model-test modules from a cold process and writes MODEL_LINE_COVERAGE.md. Standard-library trace line events are intersected with recursive bytecode source-line tables. Every model file is inventoried, including completely unexecuted modules. Fixtures validate partial/unexecuted modules, aggregate arithmetic, unsuccessful suites and trace-hook recovery after deliberate recursion-limit tests.

Actual measurement: 659/661 executable model lines, 99.70%. Uncovered lines are model/search.py:189 and model/validation.py:16. See the separate root MODEL_LINE_COVERAGE.pdf for method and limitations. No minimum percentage is specified by the assignment. Requirements coverage is a separate report; neither metric proves the other. A failed suite returns nonzero and does not publish a new successful coverage report.

Standard library debugging

This exact local pdb procedure was executed successfully:

```
python3.12 -B -m pdb main.py
```

At the debugger prompt, enter:

```
b model/service.py:56
c
```

At pim>, enter:

```
create PLAIN_TEXT text="debug note"
```

Execution stops after the candidate is built. At (Pdb), use p candidate to inspect it, then c. Back at pim>, enter exit and d. When pdb offers a restart, enter q. Breakpoint 56 belongs to this delivered source snapshot; adjust it after editing the file. Normal application commands and debugger commands belong to different prompts.

IDE instructions

VS Code is the recommended IDE; it was not used as evidence of a successful Windows run. Open the source folder, select the intended Python 3.12 interpreter, and optionally install Microsoft's Python and Python Debugger extensions. Select the supplied PIM terminal debugger configuration in .vscode/launch.json. Its working directory is the source root and its console is the integrated terminal, which is needed for input(). Set a breakpoint in PIMService.create. These editor extensions are development tools, not runtime dependencies. The VS Code procedure remains unexecuted; the standard-library pdb procedure above is the verified debugging route.

Windows setup and commands

Status: instructions provided, actual Windows execution outstanding.

Install/select Python 3.12 for Windows. The locally verified patch is 3.12.6; record the actual Windows patch and OS build when testing. Extract the source to a writable directory, for example C:/Work/Group149_PIM. In PowerShell:

```
Set-Location "C:\Work\Group149_PIM"
py -0p
py -3.12 --version
py -3.12 -X utf8 -B main.py
```

If py is unavailable, use the installed Python executable's full path with the same remaining arguments. Confirm the interpreter version rather than assuming the python command selects the intended version.

```
py -3.12 -B -m unittest discover -s tests -p "test_*.py" -v
py -3.12 -B tools/report_model_coverage.py --output "docs/evidence/windows_model_line_coverage.md"
py -3.12 -B -m pdb main.py
```

The separate Windows coverage filename preserves local macOS evidence. Repeat the pdb commands on the previous page; these Windows commands are not claimed verified. When using Windows Terminal, ensure text is legible in a Unicode-capable font. -X utf8 is included in the launch command for text I/O.

Inside the application, forward-slash Windows paths avoid extra escaping:

```
save "C:/Work/PIM data/records.pim"
save "C:\\Work\\PIM data\\records.pim"
```

The directory must already exist and be writable. Quotes protect spaces; literal backslashes inside quotes must be doubled. The program does not create missing directories. Refusing overwrite preserves the existing file and dirty state. Concurrent external writers and power-loss recovery are outside scope.

Windows release acceptance checklist

Use docs/WINDOWS_VERIFICATION_CHECKLIST.md as the recordable checklist. No item below has been executed on Windows:

- Record Windows build, Python patch, tester, date, commands and source snapshot.

- Run the full suite and model measurement without third-party runtime packages.

- Run offline; start empty and explicitly load samples/mixed.pim.

- Check all four types, Unicode/paths/mobile/date rules and Tasks 2/7 becoming 7.

- Check symbol-only grammar, escaping, full invalid-branch validation and recovery.

- Exercise duplicate/delete/overwrite confirmation, no-op/failed edits and ID history.

- Check saved criteria across edits/loads, all s/d/c branches and failed-save stops.

- Check invalid files, actual permissions and closed-handle replacement.

- Check input Ctrl-C and dirty EOF: stderr warning/status 1/no autosave.

- Execute launch/debug instructions and retain genuine results.

Do not mark platform acceptance complete based on macOS tests, mocks or a documented command. Update the manuals/reports only after genuine Windows results.

Source maintenance and handoff

The .pim root contains exactly format, version and records. Version is integer 1, format is PIM, and records contain id/type plus exactly their business fields. Storage rejects duplicate JSON keys/IDs, unsupported schema, invalid types and non-canonical values. Save encodes fully, stages bytes in the destination directory, flushes/closes, then replaces. Service clears dirty only after successful save/load. Load validates the entire file before replacing records, retains criteria and never lowers session high-water.

IDs/type are immutable. Creation validates before allocation and duplicate confirmation. Updates validate a complete candidate, then commit only if values change. Deletion never lowers the session counter. A new session reconstructs history only from loaded IDs; saved files do not contain the old counter.

Use samples/design_scenario.pim for the design's exact Task 2/7 example, samples/mixed.pim for all four types and samples/empty.pim for an empty collection. All sample identities/content are illustrative. Never auto-load them at startup.

Original SRS v1.3 and merged design v1.1 sources are preserved outside this source tree. docs/specifications contains handoff copies with the user-approved permanent-EOF clarification and a precise change log. No other behaviour or architecture is redesigned. Alarm notifications remain excluded under the provisional instructor-dependent assumption.

Required Implementation files are source, this manual, the User Manual, the implementation MP4 (maximum four minutes) and the root Requirements Coverage Report. Formal Testing remains separate: automated model tests and the root Model Line Coverage Report. README is supplementary.

The actual MP4, real Windows run and official declaration with genuine agreed contributions/signatures remain student actions. The AI assistance summary supports the official course form; it does not replace or sign it. Separate presentation slides/video and final ZIP checks remain course submission tasks. No commit, push or publication is part of this handoff.